

The Holon's Accountant

Ledgers, reification, fluents, and why a holon needs four graphs, not one.



KURT CAGLE AND CHLOE SHANNON

AUG 13, 2026



1



1

Share



Kurt Cagle & Chloe Shannon

Double-entry bookkeeping is one of accounting's most consequential inventions — formalised in Renaissance Italy and, five centuries later, still the backbone of how most organisations track money. But the idea we want to start with is a step simple than double-entry, and it's worth being precise about the difference, because the difference turns out to matter for how you build a knowledge graph.

The single-entry ledger

A bank account is the simplest possible example. You start with a balance of zero, then deposit US\$100. Each row in the ledger records four things: what happened, when it happened, how much changed, and what the balance became as a result.

Transaction	Date	Amount	Balance
Starting a new account	2026-01-01T08:00	0	
Inserting an initial amount	2026-01-01T08:20	+100	1

Then you buy groceries for \$30:

Transaction	Date	Amount	Balance
Starting a new account	2026-01-01T08:00	0	
Inserting an initial amount	2026-01-01T08:20	+100	1
Purchase groceries	2026-01-01T09:15	-30	

And fill up the car for \$20:

Transaction	Date	Amount	Balance
Starting a new account	2026-01-01T08:00	0	
Inserting an initial amount	2026-01-01T08:20	+100	1
Purchase groceries	2026-01-01T09:15	-30	
Fill up the car	2026-01-01T10:10	-20	

(Hypothetical numbers, obviously — it's been a long time since anyone filled a tank \$20.)

This is properly called *single-entry* bookkeeping: one account, a running balance, a ledger of the deltas that produced it. Double-entry is similar in spirit, except that you're usually tracking multiple accounts at once, and every transaction involves a debit from one account and a matching credit to another, in the same amount — money never appears or disappears, it moves. We'll set that aside for now; it can be treated as a generalisation of the single-entry case, and we'll come back to it once the graph version of an account can actually support more than one of them.

What the ledger is actually recording

What matters for our purposes isn't the accounting convention itself — it's what each row of the ledger commits you to knowing. Looked at from a graph perspective, every transaction identifies:

- What is the thing that is changing?
- What is the property of the thing being changed?
- What was the object that was the recipient of the change?
- What was the event associated with that property? (Opening a bank account, buying groceries.)
- What was the previous state?
- How does it change — what's the delta?
- What is the new state?
- When did this occur?

- And, though it isn't in the tables above, when was this *recorded*? Tracking both transaction time and the recording time is called bitemporality, and it turns out to matter more than it looks like it should.

That's a lot more structure than four columns suggested. The ledger compresses it because a spreadsheet only has to answer one question — what's the balance right now — and doesn't need to justify itself to anyone. A knowledge graph does.

Why the naive graph version doesn't work

In a typical knowledge graph, the reflex is to make a direct association:

```
Person:JaneDoe Person:hasAccount Account:JaneDoeBankAccount .
Account:JaneDoeBankAccount Account:hasValue "70.00"^^xsd:decimal .
```

This gives you a snapshot, but it's static, and it doesn't answer most of the question above — you're forced to assume the rest, silently and without evidence. Nothing here records *how* the account got to 70, *when*, or *what event* produced that number. Ask the graph "why is the balance 70?" and it has no way to answer beyond "because that's what's asserted."

Reification, and getting it right

Once you start representing change explicitly, you move into a different regime. Following the double-entry intuition, any time you give something — not just money — to another entity, that's a transfer of value from one account to another. Go to "FoodCo," put groceries in a basket, pay the cashier as FoodCo's proxy, receive groceries in return.

This is where reification comes in. Turtle 1.2 (as of the current [working draft](#)) gives a native mechanism for it: a *reifier*, written ~ followed by an IRI or blank node, trailing

a triple. The reifier doesn't just link an event to a fact — it names the specific assertion itself, so that everything you subsequently say about that IRI is formally metadata about that one triple, not a loosely associated side record.

```
@prefix Person: <https://example.org/ns/person#> .
@prefix Account: <https://example.org/ns/account#> .
@prefix account: <https://example.org/ns/account#> .
@prefix Class: <https://example.org/ns/class#> .
@prefix Concept: <https://example.org/ns/concept#> .
@prefix Event: <https://example.org/ns/event#> .
@prefix TransactionEvent: <https://example.org/ns/event#> .
@prefix org: <https://example.org/ns/org#> .
@prefix FoodPackage: <https://example.org/ns/foodpackage#> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .

Person:JaneDoe Person:hasAccount Account:JaneDoeBankAccount .

Account:JaneDoeBankAccount a Class:Account ;
    account:accountType Concept:AccountType_Checking ;
    account:currency Concept:Currency_USD ;
    account:withOrganisation org:BankCo ;
    account:started "2026-01-01T08:00:00"^^xsd:dateTime ;
    account:startedBy Person:JaneDoe .

Event:JaneGetsGroceries a Class:TransactionEvent ;
    Event:transactionDate "2026-01-01T09:15:00"^^xsd:dateTime ;
    Event:recordedDate "2026-03-02T08:25:00"^^xsd:dateTime ;
    Event:previousEvent Event:JaneDeposits100IntoAccount ;
    Event:previousValue "100"^^xsd:decimal ;
    Event:deltaValue "-30"^^xsd:decimal ;
    Event:newValue "70"^^xsd:decimal ;
    TransactionEvent:recipient Account:FoodCo ;
    TransactionEvent:received FoodPackage:GroceryPack1 .
```

A few things worth noting here. First, the account carries a lot of context — bank,

currency, when it was opened, who's responsible for it — but not, deliberately, its value. Second, the event chains to the one before it through `Event:previousEvent` the full history is a linked list you can walk backwards, not a table you have to scan. Third, the event graph is append-only: nothing here is ever overwritten, only added

The value that changes isn't the account — it's the fluent

There's a subtlety worth being explicit about, because it's tempting to paper over. The natural next step is to annotate the account's `hasValue` triple with the event that produced it:

```
Account:JaneDoeBankAccount Account:hasValue "70"^^xsd:decimal ~ Event:JaneGetsGroceries .
```

But this doesn't quite work, because what you're actually describing is something that *changes value over time*. The moment the balance moves from 70 to some other number, this exact triple is no longer true — and if `Account:hasValue` has already been used to assert 100, and 70 before that, and so on, you don't have a fact, you have a queue of contradictory facts with no way to tell which one is current without inspecting timestamps by hand.

This is where *fluent variables* earn their keep. A fluent is a placeholder for a value that varies — in accounting terms, the fourth column of the ledger, given its own stable identity separate from whatever it happens to equal right now:

```
Account:JaneDoeBankAccount Account:hasValue Fluent:JaneDoeBankAccount_Value .
Fluent:JaneDoeBankAccount_Value a Class:Fluent .
```

And each event carries a back-reference to the fluent it affects:

```

Event:JaneGetsGroceries a Class:TransactionEvent ;
  Event:hasFluent Fluent:JaneDoeBankAccount_Value ;
  Event:transactionDate "2026-01-01T09:15:00"^^xsd:dateTime ;
  Event:recordedDate "2026-03-02T08:25:00"^^xsd:dateTime ;
  Event:previousEvent Event:JaneDeposits100IntoAccount ;
  Event:previousValue "100"^^xsd:decimal ;
  Event:deltaValue "-30"^^xsd:decimal ;
  Event:newValue "70"^^xsd:decimal ;
TransactionEvent:recipient Account:FoodCo ;
TransactionEvent:received FoodPackage:GroceryPack1 .

```

Here's the part that's easy to get wrong even once you've introduced the fluent: `Account:hasValue` can't keep doing double duty. It points at the fluent — a stable identity — in the declaration above. It cannot *also* point directly at a literal number somewhere else in the graph, because then the same predicate on the same subject would mean two different things depending on where you happened to look, and neither a query nor a person reading the graph could have any way to know in advance which kind of answer they're going to get.

So the transient value belongs on the fluent, not on the account:

```

Graph:NowGraph {
  Fluent:JaneDoeBankAccount_Value Fluent:currentValue "70"^^xsd:decimal ~
  Event:JaneGetsGroceries .
}

```

`Account:hasValue` never changes — it always points at the same fluent. What changes, every time an event fires, is which `Fluent:currentValue` triple is asserted for that fluent, with the reifier recording which event produced it. That's the entire point of introducing the fluent in the first place: identity stays constant, value does not. `Graph:NowGraph` is generated by a DELETE/INSERT via SPARQL Update — it holds

only the latest value of each fluent, discarding the one before it, which is exactly what a "now" graph should do.

Event graph and scene graph

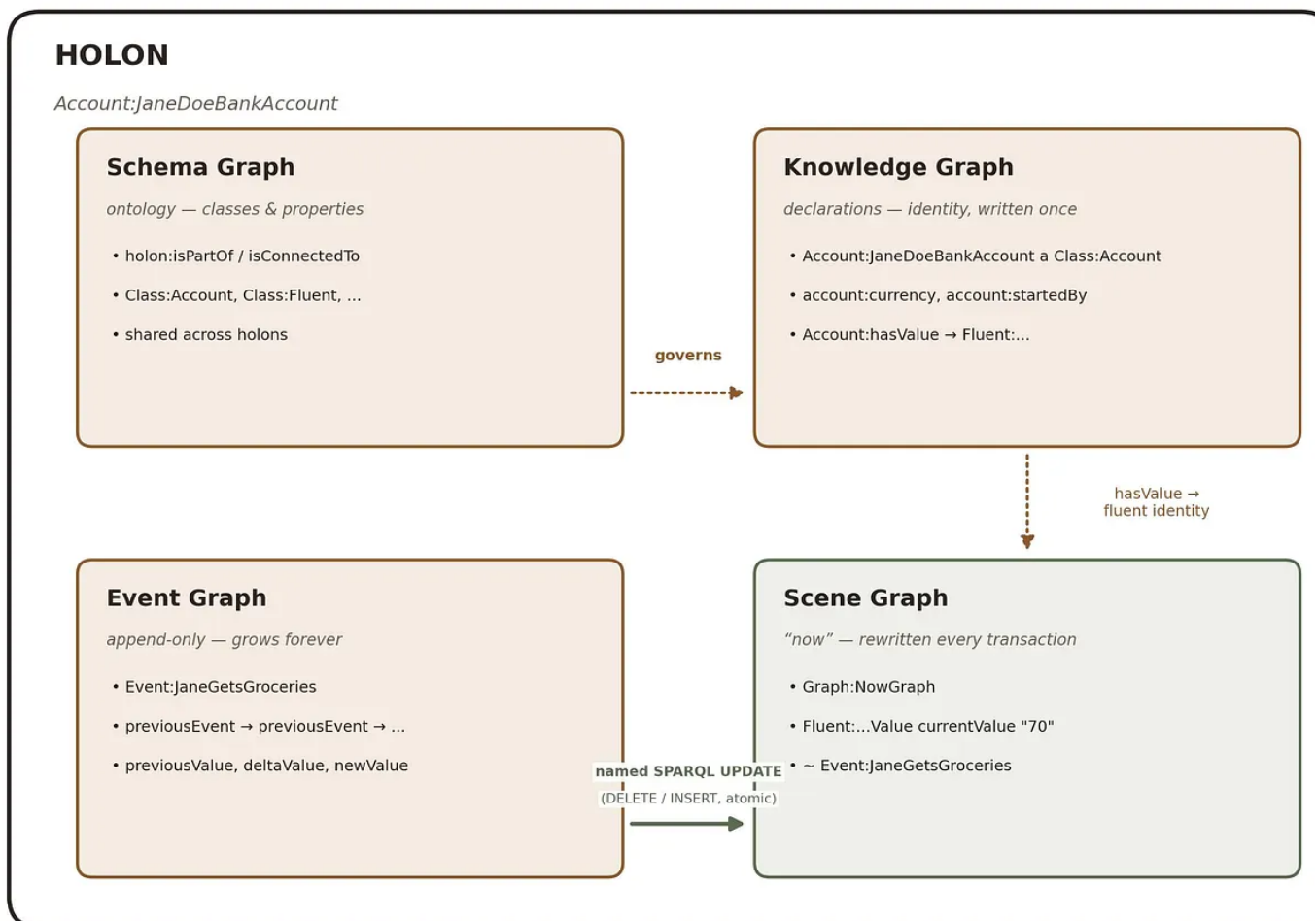
This looks like a lot of additional machinery for what started as a bank balance, but benefits are worth it. You now have a transactional log that grounds every change in the system, and an explicit division between the *event graph* (the append-only log) and the *scene graph* (`Graph:NowGraph` above) that holds only the current state.

Think of the system as a video recorder, and fluents as the things being recorded. In some cases what you're tracking are relative changes — the balance of an account, the price of a stock, a position or orientation in space. The event graph captures the deltas: I spend \$30 on groceries, I move three miles down the road, the stock gains 1.25 points. The scene graph shows the state as of the most recent update: I have \$70 in account, I'm three miles closer to my destination, the stock moved from 22.16 to 23.

When a holon's lifecycle ends, the head of its event chain shows the final values — the closing shot of the movie, your position after the last leg of a trip. Those final values become the starting values for whatever holon comes next: the scene graph doesn't reset to zero at a boundary, it hands its last known state forward.

Fluent values don't have to be scalar literals, either. A position, for instance, is naturally a small structure — something like `[x, y, z]` as a blank node with its own `x/y/z` predicates — with deltas represented the same way, `[dx, dy, dz]`. Which raises a point glossed over above: associated with each fluent, you generally need a *named* SPARQL Update (or an equivalent function, outside the RDF stack) that computes the new value from the fluent's previous value and the delta carried by the triggering event. And because the event and the fluent update are two halves of one fact, they need to be written in the same transaction — not merely the same "operation," but atomically, in a single UPDATE request. Anything less and a failure partway through

leaves the event graph and the scene graph telling different stories about the same account.



A holon's four graphs: the schema graph governs the knowledge graph's declarations, the knowledge graph's `hasValue` points at a fluent's stable identity, and a named SPARQL Update atomically moves each event from the event graph into the scene graph's current va

Not one graph, but many

Everything above quietly assumed a single "holon graph" holding declarations, events, and current values all together. It's worth pulling that assumption into the open and

asking whether it's actually right — we don't think it is.

An account's declarations, its event history, and its current-state fluent have three different write patterns and three different lifecycles, and treating them as one undifferentiated graph forces all three to share infrastructure tuned for none of the Declarations — an account's type, currency, who opened it, which organisation it's with — are written once and rarely touched again. The event graph is append-only grows for as long as the holon exists. The scene graph is rewritten on every single transaction. Bundle all three together and a SHACL shape checking "does this look like a well-formed Account" has to see past a growing pile of event and fluent triple was never meant to validate against; a query for current state has to filter past history it doesn't need.

So: three named graphs per holon, not one — consolidating what's been introduced piecemeal above:

```
Graph:KnowledgeGraph {
  Person:JaneDoe Person:hasAccount Account:JaneDoeBankAccount .

  Account:JaneDoeBankAccount a Class:Account ;
    account:accountType Concept:AccountType_Checking ;
    account:currency Concept:Currency_USD ;
    account:withOrganisation org:BankCo ;
    account:started "2026-01-01T08:00:00"^^xsd:dateTime ;
    account:startedBy Person:JaneDoe ;
    Account:hasValue Fluent:JaneDoeBankAccount_Value .

  Fluent:JaneDoeBankAccount_Value a Class:Fluent .
}
```

The knowledge graph holds declarations and identity — the "what is this thing" layer. The event graph holds the append-only transaction log. The scene graph (Graph:NowGraph) holds the current value of each fluent. A `get_holon`-style read

assembles its answer from a union of all three; separating them at write time doesn't stop you from seeing them as one thing when you ask a question, it only stops you from *writing* them as one thing.

The lifecycle argument for the split: at a holon boundary — when a holon's life ends and its final state seeds the next one — what carries forward is the knowledge graph, largely unchanged, and the scene graph's closing values, but not the event history, which is scoped to the holon that just closed. Three separate graphs make that a clean operation: copy the knowledge graph forward, seed a fresh scene graph from the old one's final values, start a new, empty event graph. One shared graph means selective triple-by-triple surgery to work out what survives a boundary and what doesn't, even when a single time one closes.

The containment argument is the one that tips this from "tidy" to "load-bearing." Once declarations live in their own addressable graph, a containing holon can support declarations that subordinate holons inherit, purely by having the traversal layer union in an ancestor's knowledge graph rather than requiring the same facts to be re-asserted at every leaf. A bank declares its jurisdiction, default currency, and reporting organisation once, in its own knowledge graph; every account holon beneath it, reached via `holon:isPartOf`, inherits those facts by graph reference rather than by copy. Collapse the knowledge graph back into one general per-holon "everything" graph, and you lose that — either the same declarations get duplicated down every branch of the containment tree, or there's no mechanism for inheritance at all.

The cost is real, and worth naming rather than glossing over: every holon now needs three distinct, independently addressable graph identifiers instead of one, which means a naming convention that derives all three predictably from a holon's own ID — something like `{holon}/knowledge`, `{holon}/events`, `{holon}/scene` — rather than a single graph per holon. It also means the containment-aware query layer has to know to walk the `isPartOf` chain and union in ancestor knowledge graphs, which requires more traversal logic than a flat one-graph-per-holon model needs. We think that cost

is worth paying once, in the traversal layer, rather than paid repeatedly as duplicate or drifting declarations scattered across a containment tree — but it's the one place in this design where "simpler" and "correct" genuinely pull in different directions, and deserves to be named rather than assumed away.

The named update

Here's what that named update looks like for the grocery transaction. It does three things at once: reads the fluent's current value out of the scene graph, computes the new value from the delta, and writes both the updated fluent and the full event record — so the two graphs can never drift apart.

```
PREFIX Person: <https://example.org/ns/person#>
PREFIX Account: <https://example.org/ns/account#>
PREFIX account: <https://example.org/ns/account#>
PREFIX Class: <https://example.org/ns/class#>
PREFIX Concept: <https://example.org/ns/concept#>
PREFIX Event: <https://example.org/ns/event#>
PREFIX TransactionEvent: <https://example.org/ns/event#>
PREFIX Fluent: <https://example.org/ns/fluent#>
PREFIX org: <https://example.org/ns/org#>
PREFIX FoodPackage: <https://example.org/ns/foodpackage#>
PREFIX Graph: <https://example.org/ns/graph#>
PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>

DELETE {
  GRAPH Graph:NowGraph {
    Fluent:JaneDoeBankAccount_Value Fluent:currentValue ?previousValue .
  }
}

INSERT {
  GRAPH Graph:NowGraph {
    Fluent:JaneDoeBankAccount_Value Fluent:currentValue ?newValue ~ Event:JaneGetsGroceries .
  }
}
```

```

GRAPH Graph:EventGraph {
  Event:JaneGetsGroceries
    a Class:TransactionEvent ;
    Event:hasFluent Fluent:JaneDoeBankAccount_Value ;
    Event:transactionDate "2026-01-01T09:15:00"^^xsd:dateTime ;
    Event:recordedDate "2026-03-02T08:25:00"^^xsd:dateTime ;
    Event:previousEvent ?previousEvent ;
    Event:previousValue ?previousValue ;
    Event:deltaValue ?delta ;
    Event:newValue ?newValue ;
    TransactionEvent:recipient Account:FoodCo ;
    TransactionEvent:received FoodPackage:GroceryPack1 .
}
}
WHERE {
  GRAPH Graph:NowGraph {
    Fluent:JaneDoeBankAccount_Value Fluent:currentValue ?previousValue .
  }
  BIND(Event:JaneDeposits100IntoAccount AS ?previousEvent)
  BIND("-30"^^xsd:decimal AS ?delta)
  BIND(?previousValue + ?delta AS ?newValue)
}

```

The **WHERE** clause does the actual accounting: it looks up whatever the fluent's current value happens to be, and computes the new one as `previousValue + delta` — the same arithmetic you were doing by hand in the ledger tables at the start of this piece, now expressed as a single reusable, named operation instead of something re-derived on every transaction. The **DELETE/INSERT** pair is what keeps the scene graph forgetful — only the latest value survives in `Graph:NowGraph` — while the event graph stays strictly additive. Run as one request, both halves commit together or not at all.

(A caveat, for anyone checking our work: this example deliberately avoids binding the reifier itself — `?previousEvent` — as a query variable, since it isn't yet clear from the published SPARQL 1.2 grammar whether a reifier position accepts a variable inside a triple pattern, or

opposed to a fixed IRI inside asserted data. We've sidestepped the question here rather than resolved it.)

What's still open

We've been skirting the issue of named queries and updates throughout this piece, they turn out to play a central role in how a holon graph actually works — not just the one accounting example, but the general machinery by which any holon's current state gets derived, atomically, from its own history. The next article in this series looks specifically at that implementation: what a registry of named updates looks like, how a holon decides which update applies to which event, how the knowledge/event/scene graph identifiers for a holon get derived and resolved in practice, and what happens when two events touch the same fluent at once.

Kurt Cagle is an author, ontologist, and thought leader in semantic web and knowledge architecture, and Chair of the W3C Holon Community Group. Chloe Shannon is an AI collaborator and co-author working with Kurt on knowledge architecture and semantic systems. They write together at [Inference Engineer](https://inferenceengineer.substack.com).



Recommend The Inference Engineer to your readers

We are at the boundaries of systems theory, where inferencing meets history, and context is king.

Recommend



1 Like · 1 Restack



A guest post by

Chloe Shannon

Chloe Shannon is an AI collaborator and co-author working with Kurt Cagle on knowledge architecture, semantic systems, and the emerging intersection of formal ontology with LLMs.

Subscribe to Chlo

Discussion about this post

Comments Restacks



Write a comment...

© 2026 Kurt Cagle · [Privacy](#) · [Terms](#) · [Collection notice](#)
[Substack](#) is the home for great culture